

Initial Logic & Build Decisions

The Core Approach

I'm not going to just feed every file into a massive prompt. That's lazy, expensive, and a security nightmare for "private" data. Instead, I'm building this as an agent with specific tools. The LLM acts as the router—it decides if it needs to hit the SQL DB for numbers, the PDF search for context, or a CSV for specific business stats.

Why Gemini 2.5 Flash?

It's fast and the tool-calling is reliable. In a system like this, I need the "thinking" step to be snappy so the UI doesn't feel like it's hanging.

Data & Privacy

I'm treating the data sources as "black boxes" to the LLM until it specifically asks for something. By using function calling, the raw data stays in my backend/database environment. The LLM only sees the filtered result set I choose to send back.

Database: SQLite vs. Postgres

I'm going with SQLite.

- **Pros:** It's a single file. When the interviewers download the repo, they don't have to spend 20 minutes configuring a local Postgres instance or Docker volumes just to see the app work.
- **Cons:** Not meant for high concurrency, but for an internal analytics tool prototype, the "zero-config" win is much more important.

A. Framework: Building from Scratch

I'm ditching LangChain for the core orchestration. Building the message loop and tool-calling logic from scratch using the Gemini SDK shows I actually understand how the state and the flow work, rather than just wrapping a high-level library. If I hit a massive wall, I can fallback, but scratch is the primary path.

B. The SQL "Middle Ground"

Text-to-SQL is risky, but fixed functions are too limiting for "unseen" queries.

- **My Solution:** A "Checked Text-to-SQL" approach. The LLM generates a query, but before it hits the DB, it goes through a Python-based validator. This validator will:

1. Block destructive keywords (DROP, DELETE, UPDATE).
 2. Check against a whitelist of tables.
 3. Enforce a `LIMIT` on all queries to prevent memory exhaustion.
- This allows for flexibility without giving the LLM the keys to the kingdom.

C. Vector DB Selection

I looked at industry standards like Pinecone, Weaviate, and Milvus.

- **Choice:** ChromaDB.
- **Reasoning:** While Milvus/Pinecone are standard for massive scale, they often require external cloud keys or heavy Docker orchestration. ChromaDB is the "industry standard" for local-first, developer-friendly embedding storage. It allows me to show a full RAG pipeline (embeddings, similarity search) without forcing the interviewer to sign up for a 3rd party cloud service.

D. UI & Traceability

A "Trace View" is non-negotiable for management trust.

- **Security Implementation:** The LLM won't be the one "showing" the document. The backend will provide metadata (filename, page, snippet ID). The frontend will then pull that specific context for the user. This maintains the "Safe Input" rule while satisfying the "Explainability" requirement.

Key Assumptions

1. **Access Control Hierarchy:** I am assuming a tiered access model. The **User** and the **Backend** have full access to the source documents (for viewing and verification). However, the **LLM** is restricted—it is "masked" from the raw documents and only receives specific snippets or query results requested via tools. This prevents the model from potentially leaking the entire dataset through creative prompting.